Практическая работа №5

Строки, массивы, циклы и методы обработки данных

Работа является частью последовательного курса. Требования к самостоятельности увеличиваются от работы к работе. Исходный проект намеренно содержит незавершённые места и отдельные спорные решения. Результат оценивается не только по факту запуска программы, но и по способности объяснить алгоритм, класс, метод, проверку входных данных и изменение требования на защите.

1. Цель работы

Цель работы состоит в закреплении конкретного набора средств Java на прикладной задаче и в формировании привычки связывать программный код с алгоритмической и объектной моделью. Вам нужно получить работающий результат, но при этом сохранить понятную структуру программы, корректную обработку данных и соответствие между Activity-диаграммой, диаграммой классов и фактической реализацией.

2. Осваиваемые элементы Java:

- String и split
- массивы примитивов
- for и while
- условные операторы
- методы с параметрами и возвращаемым значением
- валидация текстового ввода

3. Предметная постановка

Необходимо обработать строку табеля одного сотрудника за рабочую неделю. В строке через точку с запятой передаются часы за пять рабочих дней. Программа должна проверить структуру данных, преобразовать значения, вычислить суммарное

и среднее время, определить количество дней с переработкой и выявить недопустимые значения.

4. Обязательные правила:

- Строка должна содержать ровно 5 значений.
- Каждое значение должно быть целым числом от 0 до 16.
- День считается переработкой, если указано более 8 часов.
- Недельная переработка равна max(0, сумма - 40).
- Ошибка одного значения делает весь табель некорректным, но сообщение должно указывать позицию.

5. Activity-диаграмма

На рисунке дан каркас алгоритма. Красные блоки TODO намеренно оставлены неполными. Вам нужно сформулировать действие точнее и после завершения кода проверить, что последовательность ветвлений совпадает с реализацией. Формулировка на диаграмме должна отражать конкретную проверку или преобразование, а не общее выражение вроде «обработать данные».

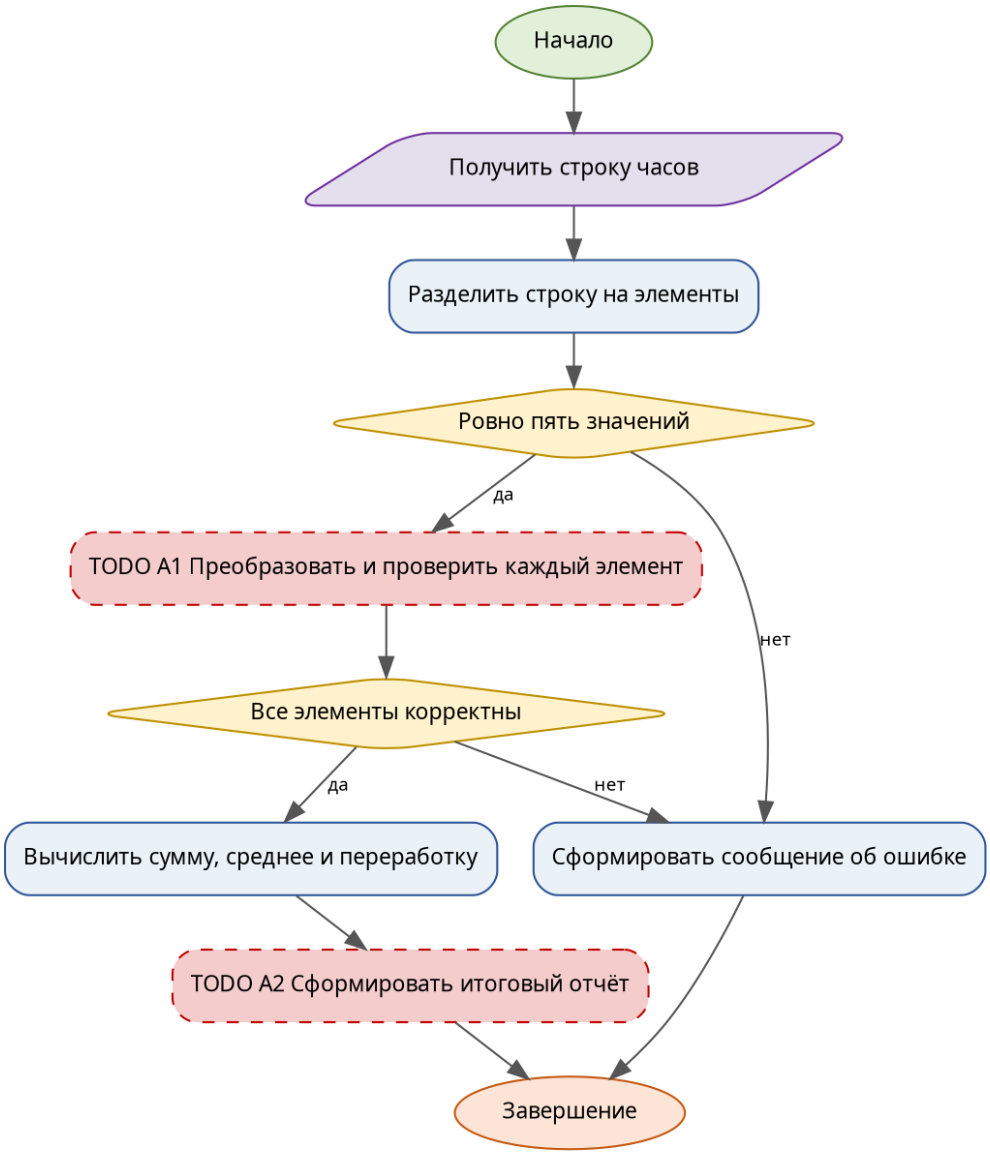


Рисунок 1 — Activity-диаграмма

6. Диаграмма классов

Диаграмма классов задаёт ожидаемую структуру решения. Элементы с пометкой TODO необходимо довести до состояния, соответствующего итоговому коду. Если в процессе выполнения студент обоснованно изменяет состав классов или зависимостей, итоговая диаграмма должна быть обновлена. Наличие класса на рисунке, которого нет в проекте, и наличие существенного класса в проекте, отсутствующего на диаграмме, считается несогласованностью.

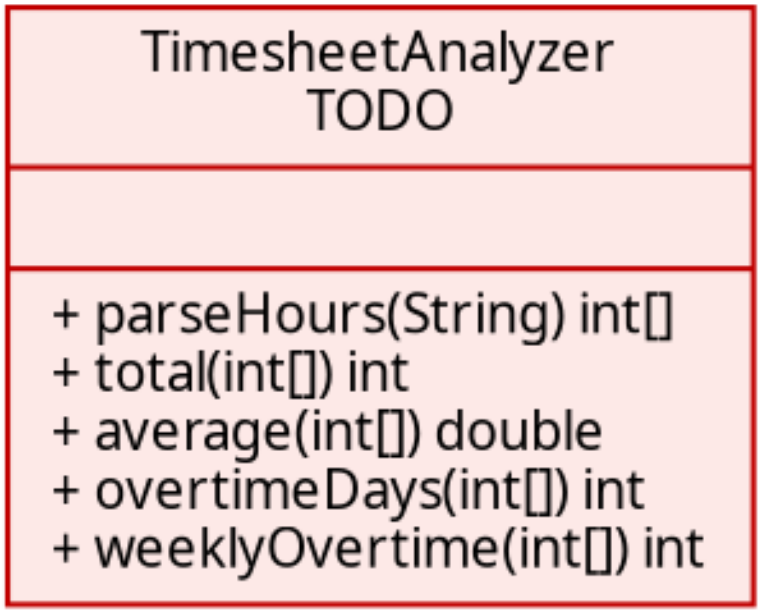


Рисунок 2 — Неполная диаграмма классов

7. Исходный проект

В каталоге *starter_project* находится компилируемый, но незавершённый Java-проект. TODO в исходном коде являются частью задания. Не требуется сохранять TODO в итоговой версии. Допускается добавление private методов и небольших вспомогательных типов, если это делает решение понятнее. Удалять *SelfCheck* до завершения работы нельзя.

8. Последовательность выполнения

1. Реализовать строгий разбор строки на пять чисел.

2. Сделать сообщение ошибки с номером позиции.

3. Реализовать вычисление суммы, среднего и количества дней переработки отдельными методами.

4. Не использовать Stream API в этой работе.

5. Добавить не менее пяти собственных проверок, включая пустую строку и значение 17.

6. Завершить Activity-диаграмму и объяснить, где находится цикл обработки массива.

9. Фрагмент, который необходимо проанализировать

Ниже приведён короткий фрагмент, который выглядит правдоподобно, но нарушает правило текущей работы. Необходимо объяснить проблему и показать исправленный подход в своём проекте или отчёте.

for (int i = 0; i <= hours.length; i++) { total += hours[i]; }

Сформулируйте конкретную ошибку этого фрагмента применительно к текущему проекту

При i = hours.length, i уйдет за границы массива, надо изменить на <.

---

10. Входные данные и граничные проверки

В *starter_project/data* находится открытый набор sample.csv. Он не является полным тестом. Студент обязан добавить собственные проверки. На защите используется другой набор входных данных, поэтому решение, подогнанное только под открытый файл, не считается завершённым.

Проверка	Что необходимо подтвердить
корректное типовое значение	ожидаемый успешный результат
нижняя граница	проверка точного включения или исключения
верхняя граница	проверка точного включения или исключения
пустое или отсутствующее значение	контролируемая ошибка
значение неверного формата	контролируемая ошибка
повтор или конфликт данных	поведение согласно правилам работы

11. Индивидуальная часть

Номер варианта вычисляется как остаток от деления двух последних цифр номера зачётной книжки на 8 плюс 1. Вариант изменяет одно локальное правило, поэтому студент должен заранее определить место в коде, где параметр или правило должно быть сосредоточено. Размножение одного и того же порога по нескольким классам усложнит защиту.

Вариант	Изменяемый аспект	Параметр
1	норма часов в день	значение или модификатор 10 точное применение определяется текстом текущего правила
2	верхняя граница часов	значение или модификатор 15 точное применение определяется текстом текущего правила
3	недельная норма	значение или модификатор 20 точное применение определяется текстом текущего правила
4	норма часов в день	значение или модификатор 25 точное применение определяется текстом текущего правила
5	верхняя граница часов	значение или модификатор 30 точное применение определяется текстом текущего правила
6	недельная норма	значение или модификатор 35 точное применение определяется текстом текущего правила
7	норма часов в день	значение или модификатор 40 точное применение определяется текстом

		текущего правила
8	верхняя граница часов	значение или модификатор 45 точное применение определяется текстом текущего правила

Запишите номер своего варианта и точное правило, которое вы реализовали

Я не знаю что такое «номер зачетной книжки», поэтому я целочислено поделил мой номер в группе на 8 и прибавил 1. Я в группе 22, т.е. мой вариант 7.

---

12. Собственные формулировки студента

Какая проверка в этой работе защищает программу от наиболее вероятной ошибки входных данных и где она расположена

Я считаю что наиболее вероятная ошибка вводных данных эт проверка числа чтобы оно было от 0 до 16. Находится внутри метода parseHours

---

Какой метод или класс содержит ключевое бизнес-правило и почему вы разместили его именно там

Я думаю ключевое бизнес правило это валидация на количество дней равное пяти. Часы переработок и часы работы за день конечно важно, но без четкого учета пятидневки ломается вообще вся логика. Поставил я эту проверку в parseHours как в основной метод валидации.

Как Activity-диаграмма отражает один конкретный участок вашего кода

Блок TODO A1 «Преобразовать и проверить каждый элемент» соответствует циклу в parseHours. Ромб «Все элементы корректны» — это проверки длины массива, parseInt и диапазона. Ветка «нет» — сообщение с номером позиции и return new int[0], ветка «да» — вызовы total, average, overtimeDays, weeklyOvertime. Блок TODO A2 «Сформировать отчёт» — вывод результатов.

Какое изменение требования в этой работе должно затронуть минимальное количество классов и почему.

Изменение макс. кол-во часов переработки в день/неделю. Это затронет лишь 1 метод.

---

13. Что необходимо сдать

- исходный проект, который компилируется без удаления файлов
- реализованные TODO и собственные методы, необходимые для решения
- обновлённую Activity-диаграмму
- обновлённую диаграмму классов
- заполненные собственные формулировки из задания
- результаты минимум пяти граничных проверок
- краткий файл DECISIONS.md с двумя решениями, которые студент считает принципиальными

14. Защита работы

На защите студент запускает проект на входных данных преподавателя, объясняет один путь исполнения по Activity-диаграмме, показывает соответствующий класс на диаграмме классов и вносит одно небольшое изменение правила. Изменение подбирается так, чтобы занимать несколько минут у студента, который понимает собственный проект, и требовало заметного переписывания у студента, который просто получил готовый код.

- объяснить назначение двух выбранных преподавателем методов
- показать обработку одного ошибочного и одного корректного входного значения
- объяснить одно граничное условие

- внести локальное изменение требования без полного переписывания программы
- повторно запустить SelfCheck и продемонстрировать отсутствие регрессии в уже работающей части

15. Критерии оценивания

Компонент	Вес	Признак завершённости
Корректность программы	35 %	правила и граничные случаи выполняются
Структура кода	20 %	ответственность не сосредоточена в одном методе
Диаграммы	15 %	совпадают с фактической реализацией
Проверки и устойчивость	15 %	ошибочные данные обрабатываются предсказуемо
Собственные объяснения и защита	15 %	студент объясняет и локально изменяет решение

16. Первоисточники для самостоятельной проверки

При выполнении работы рекомендуется сверять синтаксис и поведение библиотек с официальной документацией. Использовать вторичный источник вместо проверки API допустимо только как дополнение.

- Спецификация Java SE 25
- Учебные материалы Java Language Basics
- Классы и объекты Java

- ООП, интерфейсы и наследование